

Progress Report

Thierry Sae-Lim – Norikazu Takahashi
Okayama University
June 23, 2026

Research Objective and Goal

The final goal of the project is to implement the distributed algorithm for PCA (Principal Component Analysis) developed by our lab and test its validity based on numerical experiment.

Short-Term Goals

1. Reading the paper entitled “Fast linear iterations for distributed averaging”, by Lin Xiao and Stephen Boyd.
2. Writing a Python program to simulate the algorithm (the distributed linear iteration) in the paper (Section 1).
3. Reading the paper entitled “A Novel Iterative Method for Principal Component Analysis”, by Qiuyun Cao, Kotaro Hattori, Tsuyoshi Migita and Norikazu Takahashi.

Progress

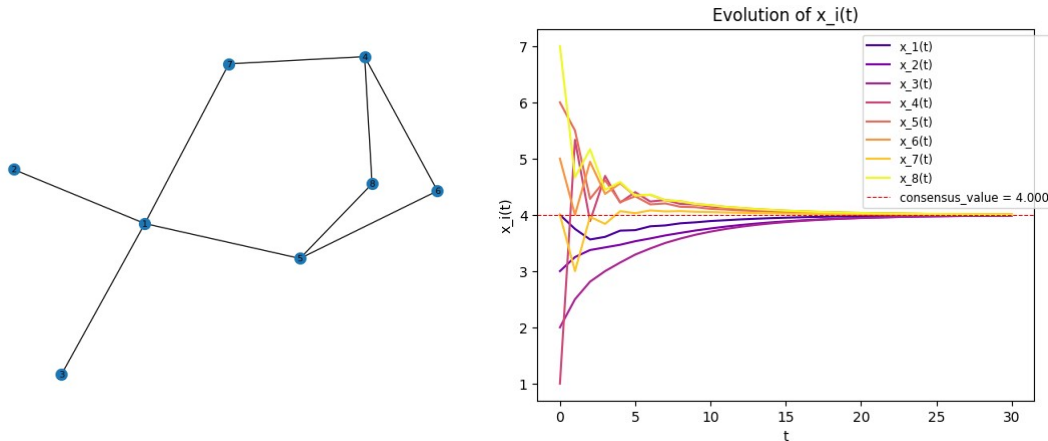
1. I read the paper, but there are some points I don't understand: the proofs of theorem 1, the section 4.1 on the constant edge weights, the section 6 on the computational methods, and the section 7 on the extensions.
2. I finished writing the program.
 - (ア) First, I defined a simple adjacency matrix that represents an undirected graph and the initial values of each nodes $x(0)$ to test with. I implemented a function to initialize the problem: it creates a dictionary to store data from the adjacency matrix and the chosen initial values. In the dictionary is stored the nodes indices (that start from 1 and not 0) as keys, and a sub-dictionary as values. In the sub-dictionary, given a node selected on the primary disctionnary, is stored the data of the node's neighbors (key: “n”), the degree of the node (key: “d”), and all the taken values of the node (key: “x”).
 - (イ) Next, I wrote a function that creates the weight matrix W from an instance of the problem based on the local-degrees weights (Section 4.2 from the paper).
 - (ウ) And finally, I implemented the distributed linear iteration:

We first need to verify the convergence conditions in the paper (Section 2).

Then we can compute and add the nexts value of each node to the data.

(工) To manipulate the data more easily, I created 2 getter functions to obtain directly the consensus value wanted, the vector x at time t .

(オ) Testing for the manually defined instance of the problem ($N=8$, $M=9$, $T=30$):



(力) Everything is working on the example adjacency matrix and the initial values I manually defined previously, but I need to test if it also work on other graphs. So I implemented a function (inspired from the generation method from the paper in Section 5.1) that randomly creates a undirected graph of N nodes and M edges:

Generating the N positions of each nodes and creating the associated complete graph. Next, finding a minimum spanning tree of the graph to ensure its connexity. Then, adding edges one by one by choosing the shortest edge that isn't in the graph yet first till the graph has M edges. Converting the generated graph to its adjacency matrix to use it next. Finally, generating randomly the vector $x(0)$ of the N initial values.

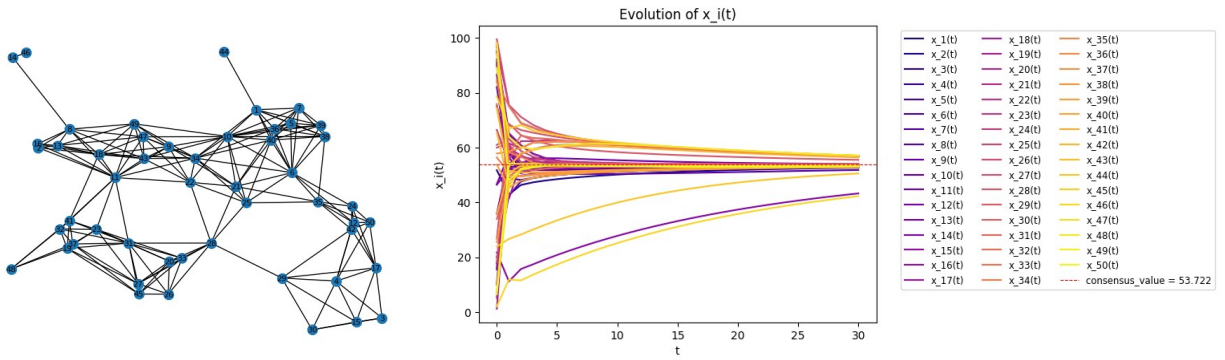
(キ) I created a function to plot the evolution of the values taken by the nodes. The plot is different depending on the parameter i :

- If i isn't set, the evolution of every agents will be shown in a single plot, it should be useful to visualize the global evolution of x over time t .
- if i is set to 0, the evolution of every agents will be shown separately.
- if i is set as a node ID, then the evolution of this agent only will be shown.

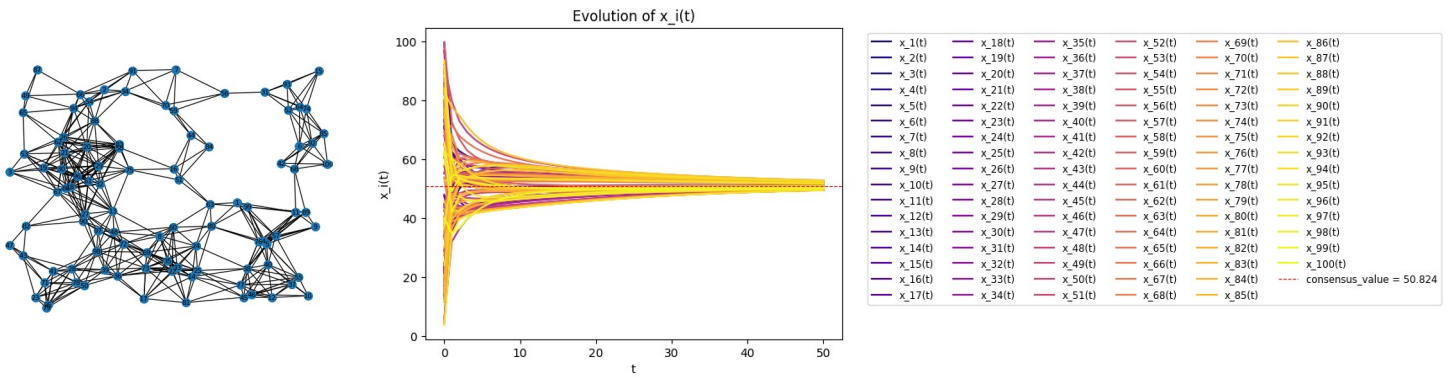
(ク) I also created a fonction to display the graph associated to an adjacency matrix, it should be useful to visualize the randomly generated graphs.

(ケ) Now I can test on randomly generated graphs, by varying the global values of N , M and the number of iteration desired T . Here are some examples:

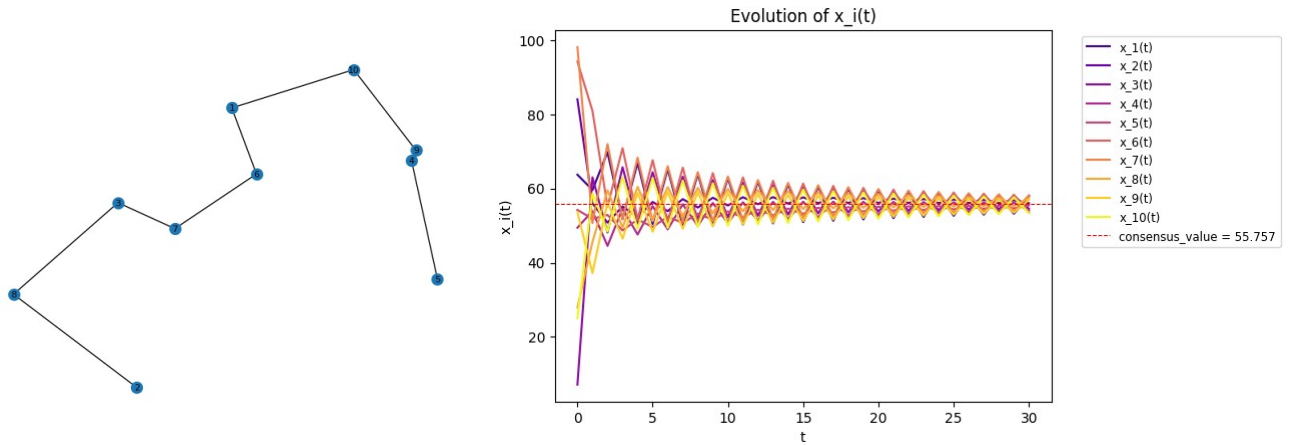
① $N=50, M=200, T=30$:



② $N=100, M=500, T=50$:



③ $N=10, M=9, T=30$:



(□) (□) The tests are complete and so is the short-term goal 2.

3. Not finished reading the paper.